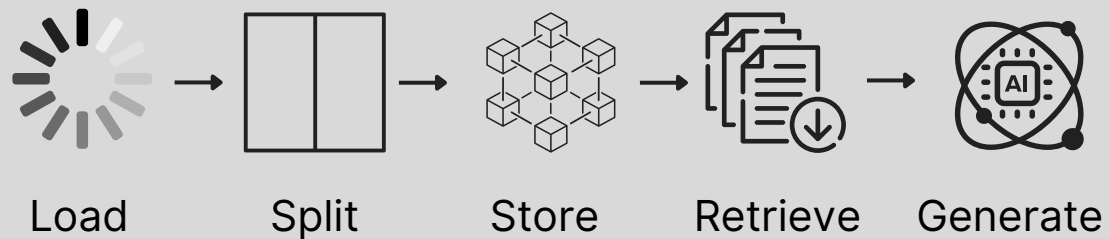


A Complete Cheatsheet



LangChain

> RAG Pipeline



> Quickstart: Loading

```
loader = PyMuPDFLoader("example_data/layout-  
parser-paper.pdf")  
docs = loader.load()
```

> Quickstart: Splitting

```
text_splitter = RecursiveCharacterTextSplitter  
                (chunk_size=1000, chunk_overlap=200)  
splits = text_splitter.split_documents(docs)
```

> Quickstart: Storing

```
vectorstore = Chroma.from_documents  
               (documents=splits,  
                embedding=OpenAIEmbeddings())
```

> Quickstart: Retrieving

```
retriever = vectorstore.as_retriever()  
  
prompt = hub.pull("rlm/rag-prompt")  
  
llm = ChatOpenAI(model_name="gpt-3.5-turbo",  
                 temperature=0)  
  
def format_docs(docs):  
    return "\n\n".join(doc.page_content for doc in docs)
```

> Quickstart: Generation

```
rag_chain = (  
    {"context": retriever | format_docs, "question":  
     RunnablePassthrough() | prompt  
     | llm  
     | StrOutputParser()}  
    .invoke("What is Task Decomposition?")
```

> Loading

Document loaders provide a "load" method for loading data as documents from a configured source

```
import bs4
```

```
from langchain_community.document_loaders import  
WebBaseLoader
```

```
bs4_strainer = bs4.SoupStrainer(class_=("post-title",  
                                         "post-header", "post-content"))
```

```
loader = WebBaseLoader(  
    web_paths=("https://lilianweng.github.io/posts/2023-  
06-23-agent/"),  
    bs_kwargs={"parse_only": bs4_strainer},)
```

```
docs = loader.load()
```

```
from langchain_community.document_loaders import  
PyMuPDFLoader
```

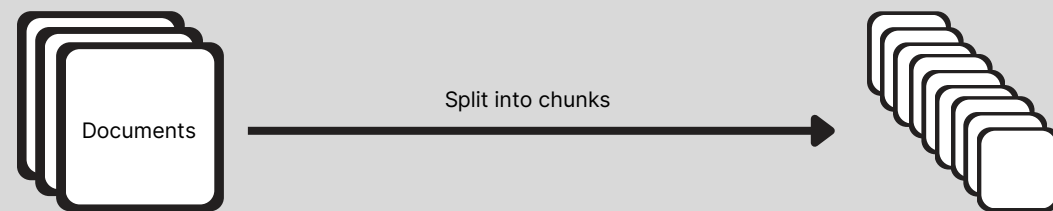
```
loader = PyMuPDFLoader("example_data/layout-  
parser-paper.pdf")  
data = loader.load()
```

```
loader = DirectoryLoader('..', glob="**/*.md",  
                        use_multithreading=True)  
docs = loader.load()
```



Split

Text splitters are tools designed to segment long pieces of text into smaller, manageable chunks while maintaining semantic coherence



```
from langchain.text_splitter import  
RecursiveCharacterTextSplitter
```

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=1000, chunk_overlap=200,  
    add_start_index=True)
```

```
all_splits = text_splitter.split_documents(docs)
```

```
python_splitter =  
RecursiveCharacterTextSplitter.from_language(  
    language=Language.PYTHON, chunk_size=50,  
    chunk_overlap=0)
```

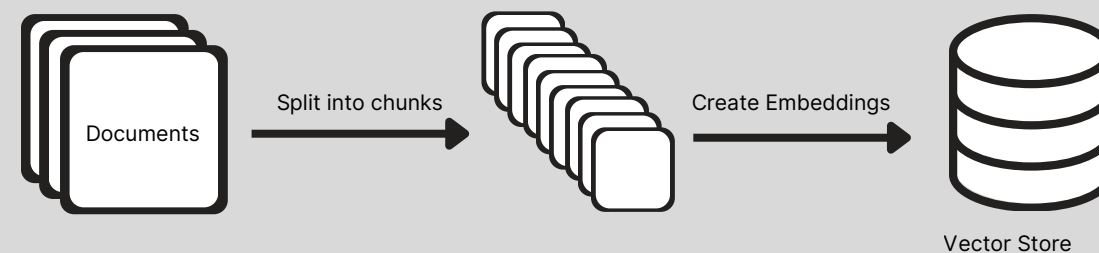
```
python_docs =  
python_splitter.create_documents([PYTHON_CODE])  
python_docs
```

```
text_splitter = SemanticChunker(OpenAIEmbeddings())  
docs = text_splitter.create_documents([File])
```



Vector Store

Vector stores are databases designed to efficiently store and retrieve high-dimensional vector embeddings of text for fast similarity search and information retrieval tasks.



```
from langchain_community.vectorstores import Chroma  
from langchain_openai import OpenAIEmbeddings
```

```
vectorstore =  
Chroma.from_documents(documents=all_splits,  
embedding=OpenAIEmbeddings())
```

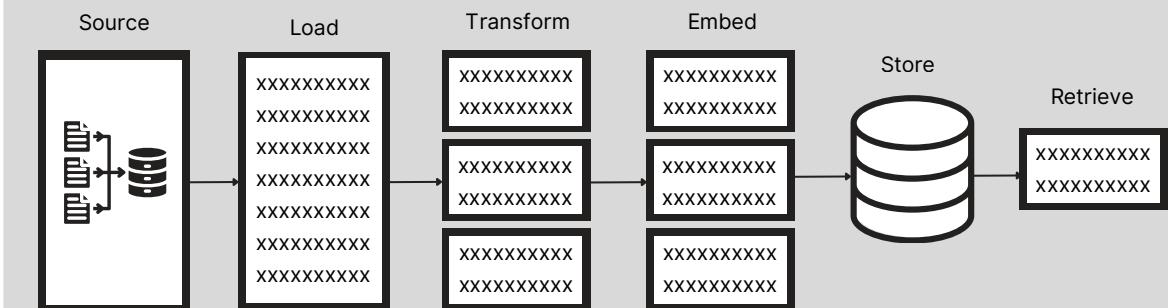
```
docs = vectorstore.similarity_search(query)  
embedding_vector =  
OpenAIEmbeddings().embed_query(query)  
docs =  
vectorstore.similarity_search_by_vector(embedding_ve  
ctor)
```

```
from langchain_community.vectorstores import Qdrant  
found_docs = await  
qdrant.amax_marginal_relevance_search(query, k=2,  
fetch_k=10)
```



Retrieve

Retrievers are components that fetch relevant information from a database based on a query, to find the most pertinent documents.



```
retriever = db.as_retriever(search_type="mmr")
```

```
retriever = db.as_retriever(  
    search_type="similarity_score_threshold",  
    search_kwargs={"score_threshold": 0.5})  
retriever = db.as_retriever(search_kwargs={"k": 3})
```

```
# parent document retriever
```

```
retriever = ParentDocumentRetriever(  
    vectorstore=vectorstore,  
    docstore=InMemoryStore(),  
    child_splitter=child_splitter)
```

```
# Time-weighted retriever
```

```
retriever = TimeWeightedVectorStoreRetriever(  
    vectorstore=vectorstore,  
    decay_rate=0.00000000000001, k=1)
```



Generate

Generators are models that produce text content, to create responses, summaries, or other forms of generated text based on input data or prompts.



```
llm = Ollama(model="llama2")
```

```
llm.invoke("Tell me a joke")
```

```
llm = HuggingFaceEndpoint(
    repo_id="mistralai/Mistral-7B-Instruct-v0.2",
    max_length=128, temperature=0.5,
    token=HUGGINGFACEHUB_API_TOKEN)
```

```
llm_chain = LLMChain(prompt=prompt, llm=llm)
```

```
print(llm_chain.run(question))
```

```
chat = ChatOpenAI(model_name="gpt-3.5-turbo",
    temperature=0)
```

```
for chunk in chat.stream("Write me a song about
    goldfish on the moon"):
    print(chunk.content, end="", flush=True)
```



Adding Sources

```
rag_chain_from_docs = (
    RunnablePassthrough.assign(context=(lambda x:
    format_docs(x["context"])))
    | prompt
    | llm
    | StrOutputParser()
)
```

```
rag_chain_with_source = RunnableParallel(
    {"context": retriever, "question":
    RunnablePassthrough()})
    .assign(answer=rag_chain_from_docs)
rag_chain_with_source.invoke("What is Task
    Decomposition")
```



Memory

```
conversation_with_summary = ConversationChain(
    llm=OpenAI(temperature=0),
    memory=ConversationBufferWindowMemory(k=5),
    verbose=True)
conversation_with_summary.predict(input="Hi, what's
    up?")
conversation_with_kg = ConversationChain(
    llm=llm, verbose=True, prompt=prompt,
    memory=ConversationKGMemory(llm=llm))
conversation_with_kg.predict(input="Hi, what's up?")
```



LangChain Expression Language

LCEL makes it easy to build complex chains from basic components

```
from langchain_core.output_parsers import
    StrOutputParser
```

```
from langchain_core.prompts import
    ChatPromptTemplate
```

```
from langchain_openai import ChatOpenAI
```

```
prompt = ChatPromptTemplate.from_template("tell me
    a short joke about {topic}")
```

```
model = ChatOpenAI(model="gpt-4")
```

```
output_parser = StrOutputParser()
```

```
chain = prompt | model | output_parser
```

```
chain.invoke({"topic": "ice cream"})
```

For more amazing cheatsheets, visit

www.analyticsvidhya.com

Follow us:

